1. Introduction

Le Lunar Hack 2.0, organisé par l'ATIA Club de la Faculté des Sciences de Tunis les 11 et 12 avril 2026, place l'intelligence artificielle au cœur de la transformation des petites et moyennes entreprises (PME). Sous le thème **« AI-Powered Smart Business Suite »**, le challenge invite les participants à concevoir une plateforme intelligente accessible qui permet à n'importe quelle PME de tirer pleinement parti de ses données, d'automatiser son marketing, d'engager ses clients de manière personnalisée et de prendre des décisions commerciales éclairées.

Dans un contexte où la majorité des PME tunisiennes et africaines disposent de données clients et de ventes mais manquent d'outils simples, intuitifs et abordables, l'IA offre une opportunité unique de démocratiser l'analytique avancée, l'automatisation et l'intelligence conversationnelle. Ce rapport technique présente la solution développée dans le cadre du hackathon, en détaillant l'architecture technique, les choix technologiques, l'implémentation des piliers retenus, ainsi que les résultats obtenus lors de la démonstration live.

L'objectif est de proposer une plateforme **complète, scalable et immédiatement utilisable** par une PME sans équipe technique dédiée, tout en respectant les contraintes du challenge (prototype fonctionnel couvrant au minimum 2 des 3 piliers, code documenté sur GitHub, vidéo démo ≤ 2 min, manuel d'utilisation ≤ 10 pages, pitch deck, etc.).

2. Problématique

Contexte Les petites et moyennes entreprises manquent souvent d'outils accessibles pour exploiter leurs données, automatiser leur marketing et engager efficacement leurs clients. L'intelligence artificielle offre aujourd'hui des leviers concrets pour combler ce fossé — mais leur intégration reste un défi majeur pour les équipes sans expertise technique avancée.

Énoncé du challenge Concevez une plateforme AI qui rend n'importe quelle PME « smart ».

La solution doit combiner :

- **analyse de données**,
- **automation marketing**,
- **intelligence conversationnelle**,
- **création de contenu**

afin de transformer la manière dont une PME comprend ses clients, communique avec eux et prend ses décisions commerciales.

Les 3 piliers à adresser (au minimum 2 piliers doivent être couverts en profondeur) :

1. **Smart Analytics Dashboard** Dashboard interactif avec insights automatiques générés par l'IA, KPIs en temps réel et recommandations actionnables basées sur les données clients et ventes.
2. **Sales Intelligence Agent** Agent conversationnel qui engage le client, qualifie ses besoins en temps réel et génère automatiquement des recommandations commerciales personnalisées selon son profil, son secteur et ses intentions.
3. **Marketing AI Assistant** Assistant capable de générer des idées de communication, du contenu marketing (textes, visuels, captions), adapté à un produit, une audience ou une campagne spécifique.

Les étapes de développement sont les suivants :

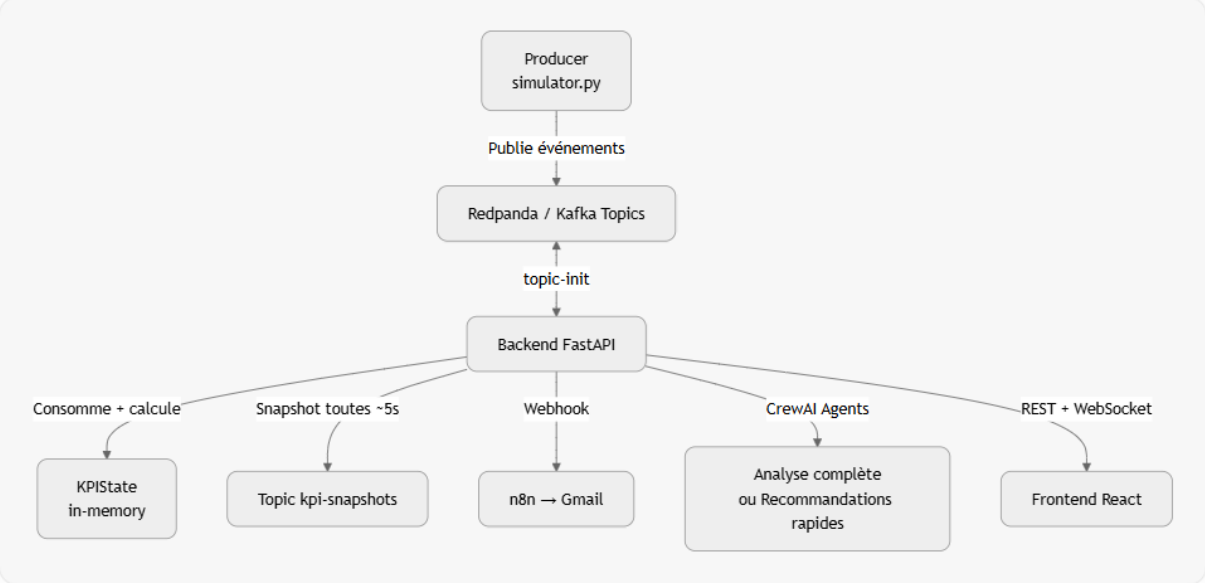
- Axe 1 : Smart Analytics dashboard

1. Workflow global (end-to-end)

Le système est conçu comme une pipeline événementielle temps réel, entièrement orchestrée via Docker Compose. Voici le flux complet du traitement des données, de la génération jusqu'à la visualisation et l'action :

1. **Démarrage de l'infrastructure**
    - docker-compose up lance l'ensemble des services :
        - **Redpanda** (broker Kafka-compatible) sur localhost:9092
        - **Redpanda Console** sur localhost:8080 (visualisation des topics et messages)
        - Script topic-init qui crée automatiquement les 9 topics Kafka : transactions, alerts, kpi-snapshots, customers, products, orders, order-items, website-behavior, conversations, marketing-campaigns
        - **Redis** sur localhost:6379 (provisionné pour futures évolutions de cache/persistance)
2. **Producteur de données simulées** (producer/simulator.py)
    - Génère en continu des événements métier réalistes (clients, produits, commandes, items, comportements web, conversations client, campagnes marketing).
    - Publie ces événements dans les topics Kafka correspondants.
    - Déclenche, selon des règles métier (VIP / churn / panier abandonné), des appels webhook vers n8n pour l'envoi automatique d'emails.
3. **Backend FastAPI** (backend/app/main.py)
    - **KafkaIngestor** : consomme tous les topics en arrière-plan (thread dédié).
    - **KPIState** : maintient un état en mémoire cohérent et calcule en temps réel tous les KPIs (CA, taux de conversion, panier moyen, taux d'abandon, etc.).
    - **SnapshotStreamProcessor** : publie toutes les ~5 secondes un snapshot des KPIs dans le topic kpi-snapshots et relit l'historique pour alimenter les graphiques du dashboard.
    - Exécute deux types d'agents IA CrewAI :
        - Pipeline complet multi-agents (/api/agent-analysis).
        - Recommandations rapides (/api/smart-recommendations).
    - Agent autonome d'emailing qui cible les clients et déclenche les webhooks n8n.
    - Expose une API REST complète + WebSocket (/ws/kpis) pour le frontend.
4. **Frontend React** (frontend/src)
    - Dashboard principal (/) : KPIs live, filtres dynamiques (produit / client / type de panier), tendances, tops produits/clients, insights IA et recommandations actionnables.
    - Page Agents (/agents) : visualisation et exécution des pipelines multi-agents CrewAI.
    - Consomme l'API REST (/api/*) et le flux WebSocket en temps réel.
5. **n8n (orchestration d'automatisation)**
    - Workflow JSON importable (N8N_API.md) : webhook send-email → nœud Gmail → réponse JSON de succès/erreur.
    - Sert de brique d'exécution fiable pour tous les scénarios d'envoi d'emails (relances, promotions, confirmations).

2. Architecture



---

3. Points d'amélioration identifiés (constats techniques)

- Les scripts consumers/kpi_consumer.py et consumers/anomaly_consumer.py sont actuellement vides (0 ligne de code). Ils constituent des points d'entrée prêts pour une consommation asynchrone plus fine ou la détection d'anomalies.
- Redis est démarré via Docker Compose mais n'est pas encore exploité dans le code applicatif. Il pourra servir ultérieurement de couche de cache distribué ou de persistance légère des états KPI.
- Incohérence de nommage des webhooks n8n : le backend appelle send-email tandis que le simulator utilise send-mails (et le dossier webhook-test/send-mails). Un alignement est recommandé pour éviter toute confusion en production.
- Le système repose actuellement sur un état en mémoire (KPIState + caches IA). Bien que performant en environnement de démo, une persistance durable (PostgreSQL ou TimescaleDB pour l'historique des commandes et snapshots) sera nécessaire pour une version production.

<u>Axe 2 : Sales Intelligence Agent</u>

1. Vue d'ensemble

L'Axe 2 constitue le **moteur local de décision assistée par IA** de la plateforme Smart Business Suite. Il est conçu pour traiter des cas d'usage métier concrets (analyse de leads, FAQ clients, évaluation de risques légaux/compliance, qualification commerciale).

Ce module repose sur une architecture hybride légère et entièrement exécutable en environnement local :

- Backend FastAPI orchestrant le flux décisionnel
- Routage heuristique déterministe (sans appel LLM systématique)
- LLM local via Ollama pour la génération structurée
- Connecteur RAGFlow pour l'ancrage documentaire (y compris documents légaux sur les droits personnels et le commerce)
- Mémoire SQLite pour la persistance des sessions et profils utilisateurs
- Frontend React offrant un chat multilingue (FR/EN/AR) avec historique complet

L'objectif est de fournir à la PME une intelligence conversationnelle fiable, contextuelle et conforme, capable de qualifier un prospect, d'identifier des risques juridiques et de générer des actions commerciales immédiatement exploitables.

2. Architecture globale

Architecture logique initialement prévue (multi-agents modulaires) :

1. Intent Agent
2. Schema Builder Agent
3. Qualification Agent
4. RAG Agent (conditionnel)
5. Recommendation Agent
6. Decision Agent
7. Response Agent

Architecture effectivement exécutée dans le code runtime (optimisée pour la performance) :

1. Réception de la requête utilisateur via l'API chat
2. **Routage heuristique déterministe** multilingue (détection instantanée de FAQ, stratégie commerciale, risque légal/compliance)
3. Activation conditionnelle du RAGFlow
4. Appel à un **SingleGenerationAgent** unifié (Ollama) qui génère une réponse JSON structurée
5. Calcul automatique du Lead Score et du Priority Score
6. Persistance dans SQLite + enrichissement du profil utilisateur
7. Réponse structurée au frontend

Flux simplifié : UI React → FastAPI → Router heuristique → (RAGFlow optionnel) → Ollama → Scoring + Structuration → SQLite → Réponse UI

Cette optimisation permet une latence très faible tout en conservant la puissance décisionnelle d'un pipeline multi-agents.

3. Composants techniques

Backend

- FastAPI (endpoints chat, gestion des sessions et messages)
- Orchestrateur central pilotant le flux complet
- Router heuristique multilingue (détection de signaux métier sans LLM)
- Client Ollama pour inférence locale
- Client RAGFlow pour retrieval documentaire
- Store SQLite (sessions, messages, profils utilisateurs enrichis)

- Modèles Pydantic pour contrats de données stricts

Frontend

- Interface chat React moderne et responsive
- Sélecteur de langue (FR/EN/AR) avec persistance des préférences
- Historique des sessions avec chargement instantané
- Affichage explicite des sources RAG pour chaque réponse

Données et scripts d'initialisation

- Scripts d'indexation automatique des documents vers RAGFlow
- Gestion du dataset (upload, chunking, nettoyage)
- Base documentaire enrichie avec documents légaux (droits personnels, droit commercial, conformité PME) afin de permettre au modèle de répondre de manière fiable aux questions réglementaires ou contractuelles.

4. Fonctionnalités clés

Fonctionnalités conversationnelles

- Chat décisionnel multilingue (FR/EN/AR)
- Réponses orientées action avec stratégie claire et priorisation
- Questions de relance automatique en cas d'informations manquantes
- Mention systématique des sources RAG lorsque utilisées

Fonctionnalités d'aide à la décision

- Calcul automatique du **Lead Score** (profil, urgence, confiance, complétude, nombre de tours)
- **Priority Score** pour priorisation opérationnelle immédiate
- Génération d'un plan d'action court (CTA + actions prioritaires)
- Brouillon automatique d'email de suivi opérationnel

Fonctionnalités RAG avancées

- Déclenchement conditionnel selon l'intention détectée, le risque légal ou la nécessité de facticité
- Récupération de chunks documentaires depuis RAGFlow
- Résolution automatique des dataset_ids (nom ou UUID)
- Construction d'un contexte RAG compact et pertinent
- Normalisation et affichage clair des sources renvoyées
- **Enrichissement spécifique** : inclusion de documents légaux (droits personnels et droit commercial) pour traiter les questions de conformité, contrats et obligations réglementaires courantes des PME.

Fonctionnalités mémoire et robustesse

- Persistance complète des échanges par session
- Reprise instantanée de sessions antérieures
- Profil utilisateur enrichi dynamiquement
- Contrats JSON stricts (Pydantic) + réparation automatique des sorties invalides (retry)

- Fallback déterministe en cas de non-conformité
- Traces détaillées par requête (latence par étape, nombre d'appels Ollama/RAGFlow)

5. API exposée (Axe 2)

Méthode	Endpoint	Description
GET	/health	État du service, environnement et statut de l'orchestration
POST	/chat	Point d'entrée principal du chat décisionnel
GET	/sessions/{user_id}	Liste des sessions d'un utilisateur
GET	/sessions/{user_id}/{session_id}/messages	Historique complet d'une session

6. Structure de la réponse fonctionnelle

Chaque appel à /chat retourne un payload JSON riche et immédiatement exploitable par le frontend :

JSON

```
{
  "response": "texte final destiné à l'utilisateur",
  "structured": { ... synthèse métier structurée ... },
  "user_profile": { "lead_score": 87, "priority_score": 92, "champs_requis": [...], ... },
  "rag_sources": [ { "document": "...", "excerpt": "...", "relevance": 0.94 }, ... ],
  "follow_up_email": "brouillon complet d'email de relance",
  "next_question": "question adaptative pour continuer la qualification",
  "metadata": { "latency_ms": 1240, "rag_used": true, ... }
}
```

7. Points d'attention et perspectives

- Le projet conserve la structure modulaire multi-agents dans le code source, mais le pipeline actif est optimisé autour d'un **SingleGenerationAgent** unifié pour maximiser les performances en environnement local.
- Le frontend exploite actuellement principalement le champ response et rag_sources ; la richesse du payload structured, user_profile et follow_up_email peut être davantage mise en valeur dans les futures itérations de l'interface.
- Absence actuelle d'une suite de tests automatisés dans le dossier tests/backend ; une couverture minimale (tests unitaires du router + tests d'intégration RAG/Ollama) est recommandée avant la démonstration live.

Cette architecture rend l'Axe 2 particulièrement adapté aux besoins des PME tunisiennes : rapidité, conformité légale intégrée et prise de décision actionable sans dépendance cloud lourde.

Axe 3 : Marketing AI Assistant

- En première partie : nous avons développé un système IA multi agents afin d'étudier le site web , les concurrents et identifier pour l'utilisateur les différentes techniques à faire pour améliorer la visibilité
- En deuxième partie , C'est la conception d'un chatbot qui fait la suggestion des stratégies marketing . tout en demandant à l'utilisateur des questions pour plus des clarifier